SUBJECT CODE & NAME: CSA17 - ARTIFICIAL INTELLIGENCE

NAME:J.ARCHANA
REGISTER NUMBER:192324294

# LAB RECORD - DAY 1

## List of Programs

1. 8-Puzzle Problem *(CO1)*
2. 8-Queen Problem *(CO1)*
3. Water Jug Problem *(CO2)*
4. Crypt-Arithmetic Problem *(CO2)*
5. Missionaries and Cannibals Problem *(CO3)*
6. Vacuum Cleaner Problem *(CO3)*
7. Breadth First Search (BFS) *(CO3)*
8. Depth First Search (DFS) *(CO4)*
9. Travelling Salesman Problem *(CO4)*
10. A* Algorithm *(CO4)*

# Program 1: 8-Puzzle Problem

**Course Outcome:** CO1

**Aim:** To write a Python program to solve the 8-Puzzle problem using the A* search algorithm with the Manhattan distance heuristic.

**Program:**

```
import heapq
 GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0)
def manhattan(state):
    dist = 0
    for i, tile in enumerate(state):
        if tile != 0:
            gi = GOAL.index(tile)
            dist += abs(i // 3 - gi // 3) + abs(i % 3 - gi % 3)
    return dist

def neighbors(state):
    z = state.index(0)
    r, c = divmod(z, 3)
    for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            n = nr * 3 + nc
            s = list(state)
            s[z], s[n] = s[n], s[z]
            yield tuple(s)

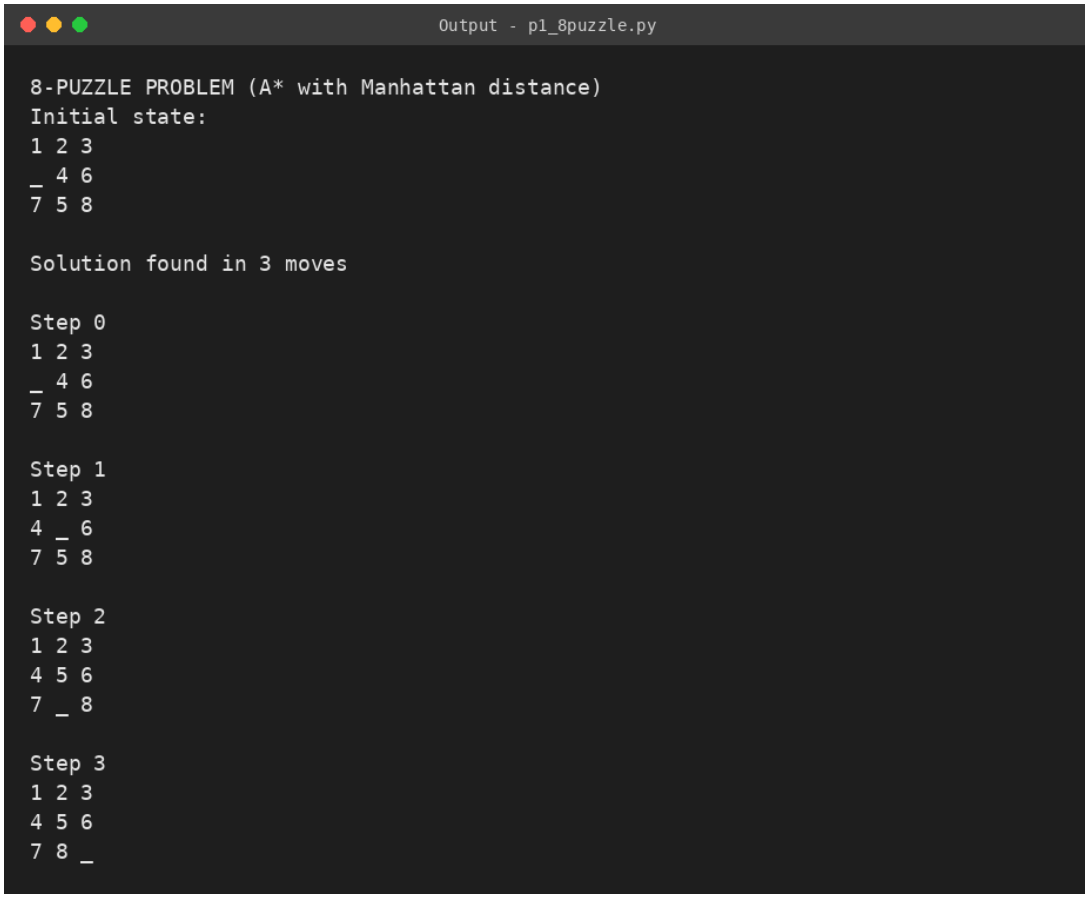

def solve(start):
    pq = [(manhattan(start), 0, start, None)]
    parent = {}
    best = {start: 0}
    while pq:
        f, g, state, par = heapq.heappop(pq)
        if state in parent:
            continue
        parent[state] = par
        if state == GOAL:
            path = []
            while state is not None:
                path.append(state)
                state = parent[state]
            return path[::-1]
        for nxt in neighbors(state):
            ng = g + 1
            if nxt not in best or ng < best[nxt]:
                best[nxt] = ng
                heapq.heappush(pq, (ng + manhattan(nxt), ng, nxt, state))
    return None


def show(state):
    for i in range(0, 9, 3):
        print(" ".join(str(x) if x else "_" for x in state[i:i + 3]))
```

```
    print()


start = (1, 2, 3, 0, 4, 6, 7, 5, 8)
print("8-PUZZLE PROBLEM (A* with Manhattan distance)")
print("Initial state:")
show(start)
path = solve(start)
if path:
    print("Solution found in", len(path) - 1, "moves\n")
    for i, s in enumerate(path):
        print("Step", i)
        show(s)
else:
    print("No solution exists")
```

**Output:**




**Result:** The Python program for 8-Puzzle Problem was executed successfully and the output was verified.

## Program 2: 8-Queen Problem

**Course Outcome:** CO1

**Aim:** To write a Python program to place 8 queens on an 8x8 chessboard such that no two queens attack each other, using backtracking.

**Program:**

```
N = 8
solutions = []

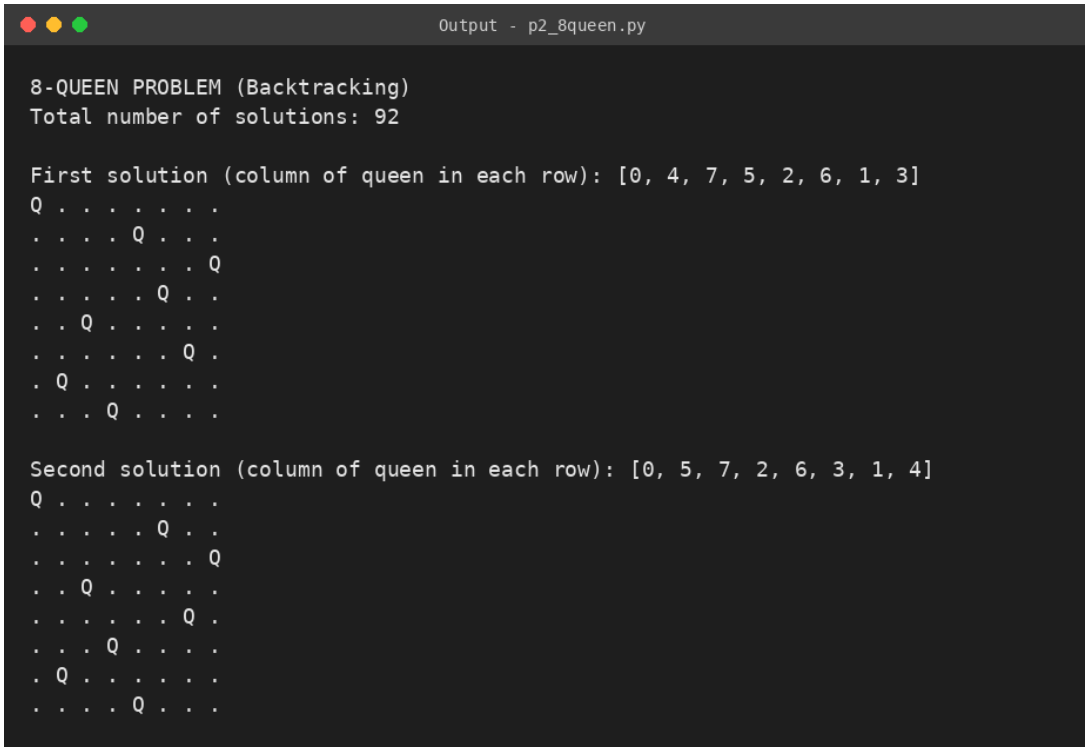

def is_safe(board, row, col):
    for r in range(row):
        c = board[r]
        if c == col or abs(c - col) == abs(r - row):
            return False
    return True


def place(board, row):
    if row == N:
        solutions.append(board[:])
        return
    for col in range(N):
        if is_safe(board, row, col):
            board[row] = col
            place(board, row + 1)
            board[row] = -1


def print_board(board):
    for r in range(N):
        print(" ".join("Q" if board[r] == c else "." for c in range(N)))
    print()


place([-1] * N, 0)
print("8-QUEEN PROBLEM (Backtracking)")
print("Total number of solutions:", len(solutions))
print("\nFirst solution (column of queen in each row):", solutions[0])
print_board(solutions[0])
print("Second solution (column of queen in each row):", solutions[1])
print_board(solutions[1])
```

**Output:**




**Result:** The Python program for 8-Queen Problem was executed successfully and the output was verified.

# Program 3: Water Jug Problem

**Course Outcome:** CO2

**Aim:** To write a Python program to measure exactly 2 litres of water using a 4-litre jug and a 3-litre jug, using Breadth First Search.

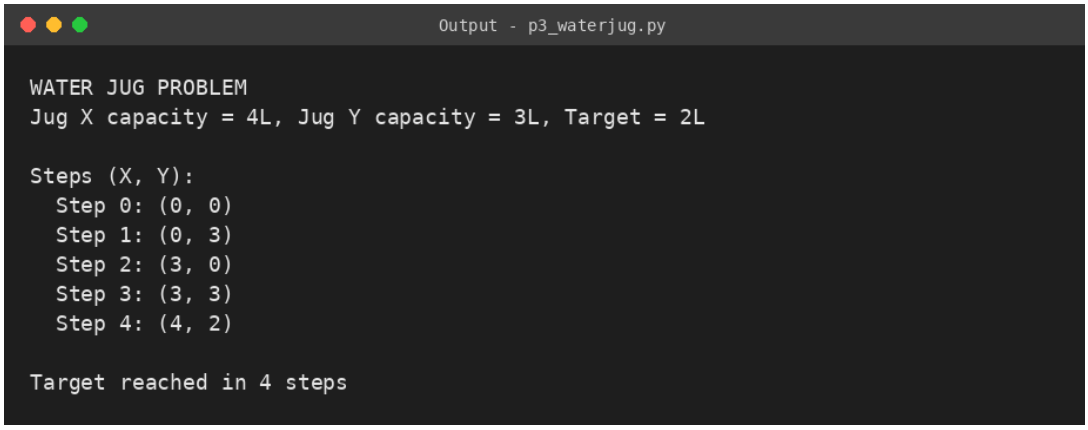
**Program:**

```
from collections import deque


def water_jug(cap_x, cap_y, target):
    start = (0, 0)
    queue = deque([(start, [start])])
    visited = {start}
    while queue:
        (x, y), path = queue.popleft()
        if x == target or y == target:
            return path
        moves = [
            (cap_x, y),                          # fill jug X
            (x, cap_y),                          # fill jug Y
            (0, y),                             # empty jug X
            (x, 0),                             # empty jug Y
            (x - min(x, cap_y - y), y + min(x, cap_y - y)),  # pour X -> Y
            (x + min(y, cap_x - x), y - min(y, cap_x - x)),  # pour Y -> X
        ]
        for m in moves:
            if m not in visited:
                visited.add(m)
                queue.append((m, path + [m]))
    return None


cap_x, cap_y, target = 4, 3, 2
print("WATER JUG PROBLEM")
print(f"Jug X capacity = {cap_x}L, Jug Y capacity = {cap_y}L, Target = {target}L\n")
path = water_jug(cap_x, cap_y, target)
if path:
    print("Steps (X, Y):")
    for i, s in enumerate(path):
        print(f"  Step {i}: {s}")
    print("\nTarget reached in", len(path) - 1, "steps")
else:
    print("No solution possible")
```

**Output:**




**Result:** The Python program for Water Jug Problem was executed successfully and the output was verified.

# Program 4: Crypt-Arithmetic Problem

**Course Outcome:** CO2

**Aim:** To write a Python program to solve the crypt-arithmetic puzzle SEND + MORE = MONEY by assigning a unique digit to each letter.

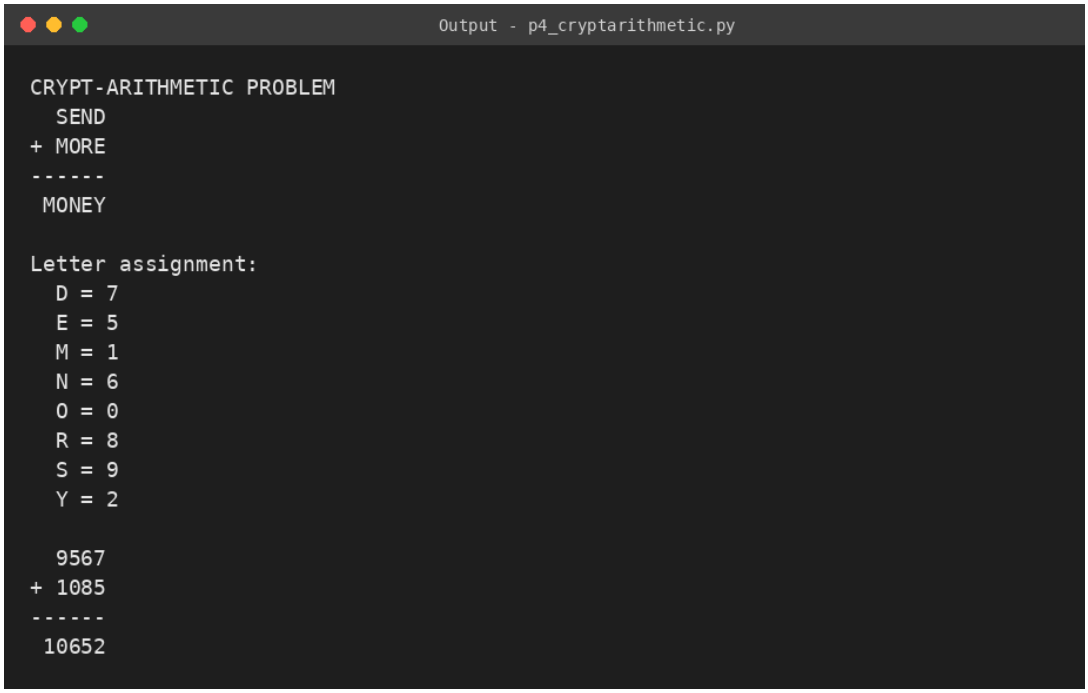
**Program:**

```
from itertools import permutations


def solve(word1, word2, result):
    letters = sorted(set(word1 + word2 + result))
    if len(letters) > 10:
        return None
    first = {word1[0], word2[0], result[0]}
    for perm in permutations(range(10), len(letters)):
        mapping = dict(zip(letters, perm))
        if any(mapping[ch] == 0 for ch in first):
            continue
        n1 = int("".join(str(mapping[c]) for c in word1))
        n2 = int("".join(str(mapping[c]) for c in word2))
        n3 = int("".join(str(mapping[c]) for c in result))
        if n1 + n2 == n3:
            return mapping, n1, n2, n3
    return None


w1, w2, res = "SEND", "MORE", "MONEY"
print("CRYPT-ARITHMETIC PROBLEM")
print(f"  {w1}\n+ {w2}\n------\n {res}\n")
sol = solve(w1, w2, res)
if sol:
    mapping, a, b, c = sol
    print("Letter assignment:")
    for k in sorted(mapping):
        print(f"  {k} = {mapping[k]}")
    print(f"\n  {a}\n+ {b}\n------\n {c}")
else:
    print("No solution found")
```

**Output:**




**Result:** The Python program for Crypt-Arithmetic Problem was executed successfully and the output was verified.

# Program 5: Missionaries and Cannibals Problem

**Course Outcome:** CO3

**Aim:** To write a Python program to move 3 missionaries and 3 cannibals across a river without the cannibals ever outnumbering the missionaries on either bank.

**Program:**

```
from collections import deque

TOTAL = 3  # missionaries and cannibals on each side initially


def valid(m, c):
    if m < 0 or c < 0 or m > TOTAL or c > TOTAL:
        return False
    if m > 0 and c > m:                # left bank
        return False
    rm, rc = TOTAL - m, TOTAL - c
    if rm > 0 and rc > rm:             # right bank
        return False
    return True


def solve():
    start = (TOTAL, TOTAL, 1)          # (M left, C left, boat on left?)
    goal = (0, 0, 0)
    boat_moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]
    queue = deque([(start, [start])])
    visited = {start}
    while queue:
        (m, c, b), path = queue.popleft()
        if (m, c, b) == goal:
            return path
        for dm, dc in boat_moves:
            if b == 1:
                nm, nc, nb = m - dm, c - dc, 0
            else:
                nm, nc, nb = m + dm, c + dc, 1
            state = (nm, nc, nb)
            if valid(nm, nc) and state not in visited:
                visited.add(state)
                queue.append((state, path + [state]))
    return None


print("MISSIONARIES AND CANNIBALS PROBLEM (BFS)")
print("State = (Missionaries left, Cannibals left, Boat side)\n")
path = solve()
for i, (m, c, b) in enumerate(path):
    side = "Left " if b == 1 else "Right"
    print(f"Step {i:2d}: Left = ({m}M, {c}C) | Right = ({TOTAL - m}M, {TOTAL - c}C) | Boat: {side}")
print("\nAll missionaries and cannibals crossed safely in", len(path) - 1, "trips")
```

**Output:**

Output - p5_missionaries.py

```
MISSIONARIES AND CANNIBALS PROBLEM (BFS)
State = (Missionaries left, Cannibals left, Boat side)

Step  0: Left = (3M, 3C) | Right = (0M, 0C) | Boat: Left
Step  1: Left = (3M, 1C) | Right = (0M, 2C) | Boat: Right
Step  2: Left = (3M, 2C) | Right = (0M, 1C) | Boat: Left
Step  3: Left = (3M, 0C) | Right = (0M, 3C) | Boat: Right
Step  4: Left = (3M, 1C) | Right = (0M, 2C) | Boat: Left
Step  5: Left = (1M, 1C) | Right = (2M, 2C) | Boat: Right
Step  6: Left = (2M, 2C) | Right = (1M, 1C) | Boat: Left
Step  7: Left = (0M, 2C) | Right = (3M, 1C) | Boat: Right
Step  8: Left = (0M, 3C) | Right = (3M, 0C) | Boat: Left
Step  9: Left = (0M, 1C) | Right = (3M, 2C) | Boat: Right
Step 10: Left = (1M, 1C) | Right = (2M, 2C) | Boat: Left
Step 11: Left = (0M, 0C) | Right = (3M, 3C) | Boat: Right

All missionaries and cannibals crossed safely in 11 trips
```

**Result:** The Python program for Missionaries and Cannibals Problem was executed successfully and the output was verified.

# Program 6: Vacuum Cleaner Problem

**Course Outcome:** CO3

**Aim:** To write a Python program that simulates a simple reflex vacuum cleaner agent cleaning a two-room environment.

**Program:**

```
def vacuum_cleaner(state, location):
    """Simple reflex agent for the two-room vacuum world."""
    rooms = dict(state)
    cost = 0
    print("Initial state:", rooms, "| Agent at room", location)
    print("-" * 50)
    for step in range(1, 5):
        if rooms[location] == "Dirty":
            print(f"Step {step}: Room {location} is Dirty  -> action: SUCK")
            rooms[location] = "Clean"
            cost += 1
        else:
            other = "B" if location == "A" else "A"
            print(f"Step {step}: Room {location} is Clean  -> action: MOVE to {other}")
            location = other
            cost += 1
        print("         State:", rooms, "| Agent at room", location)
        if all(v == "Clean" for v in rooms.values()):
            break
    print("-" * 50)
    print("Final state:", rooms)
    print("Total actions performed:", cost)


print("VACUUM CLEANER PROBLEM")
print("\n=== Case 1: Both rooms dirty, agent starts in A ===")
vacuum_cleaner({"A": "Dirty", "B": "Dirty"}, "A")
print("\n=== Case 2: Only room B dirty, agent starts in A ===")
vacuum_cleaner({"A": "Clean", "B": "Dirty"}, "A")
```

**Output:**

Output - p6_vacuum.py

```
VACUUM CLEANER PROBLEM

=== Case 1: Both rooms dirty, agent starts in A ===
Initial state: {'A': 'Dirty', 'B': 'Dirty'} | Agent at room A
------------------------------------------------
Step 1: Room A is Dirty  -> action: SUCK
         State: {'A': 'Clean', 'B': 'Dirty'} | Agent at room A
Step 2: Room A is Clean  -> action: MOVE to B
         State: {'A': 'Clean', 'B': 'Dirty'} | Agent at room B
Step 3: Room B is Dirty  -> action: SUCK
         State: {'A': 'Clean', 'B': 'Clean'} | Agent at room B
------------------------------------------------
Final state: {'A': 'Clean', 'B': 'Clean'}
Total actions performed: 3

=== Case 2: Only room B dirty, agent starts in A ===
Initial state: {'A': 'Clean', 'B': 'Dirty'} | Agent at room A
------------------------------------------------
Step 1: Room A is Clean  -> action: MOVE to B
         State: {'A': 'Clean', 'B': 'Dirty'} | Agent at room B
Step 2: Room B is Dirty  -> action: SUCK
         State: {'A': 'Clean', 'B': 'Clean'} | Agent at room B
------------------------------------------------
Final state: {'A': 'Clean', 'B': 'Clean'}
Total actions performed: 2
```

**Result:** The Python program for Vacuum Cleaner Problem was executed successfully and the output was verified.

# Program 7: Breadth First Search (BFS)

**Course Outcome:** CO3

**Aim:** To write a Python program to implement the Breadth First Search traversal of a graph.

**Program:**

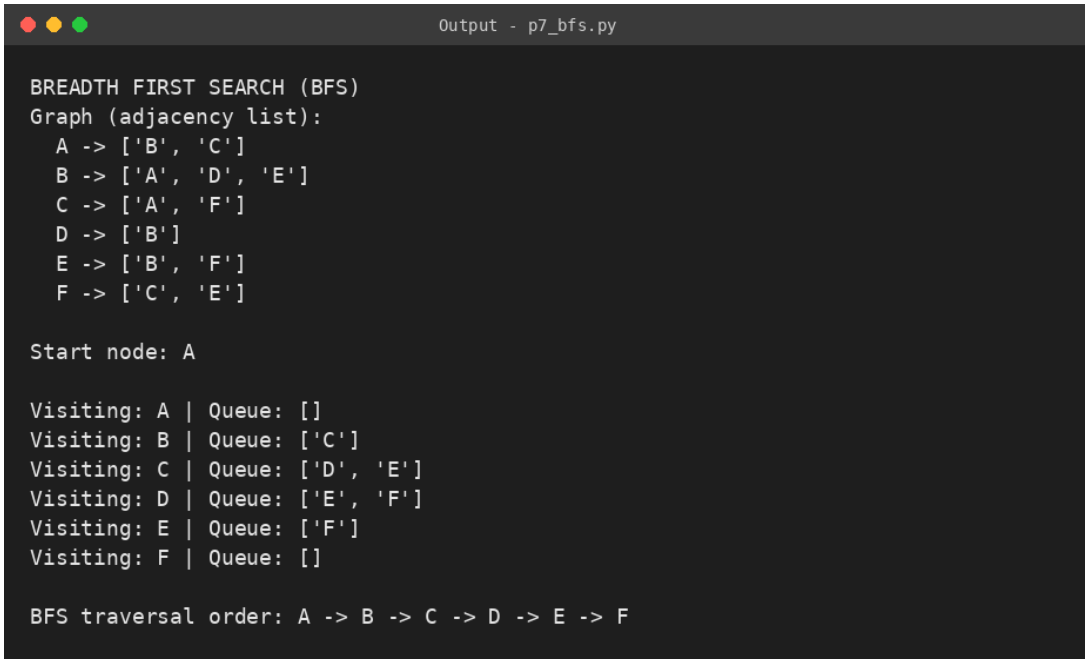
```
from collections import deque

graph = {
    "A": ["B", "C"],
    "B": ["A", "D", "E"],
    "C": ["A", "F"],
    "D": ["B"],
    "E": ["B", "F"],
    "F": ["C", "E"],
}


def bfs(graph, start):
    visited = [start]
    queue = deque([start])
    while queue:
        node = queue.popleft()
        print("Visiting:", node, "| Queue:", list(queue))
        for nb in graph[node]:
            if nb not in visited:
                visited.append(nb)
                queue.append(nb)
    return visited


print("BREADTH FIRST SEARCH (BFS)")
print("Graph (adjacency list):")
for k, v in graph.items():
    print(f"  {k} -> {v}")
print("\nStart node: A\n")
order = bfs(graph, "A")
print("\nBFS traversal order:", " -> ".join(order))
```

**Output:**




**Result:** The Python program for Breadth First Search (BFS) was executed successfully and the output was verified.

# Program 8: Depth First Search (DFS)

**Course Outcome:** CO4

**Aim:** To write a Python program to implement the Depth First Search traversal of a graph.

**Program:**

```
graph = {
    "A": ["B", "C"],
    "B": ["A", "D", "E"],
    "C": ["A", "F"],
    "D": ["B"],
    "E": ["B", "F"],
    "F": ["C", "E"],
}


def dfs(graph, node, visited, depth=0):
    visited.append(node)
    print("  " * depth + "Visiting:", node)
    for nb in graph[node]:
        if nb not in visited:
            dfs(graph, nb, visited, depth + 1)
    return visited


print("DEPTH FIRST SEARCH (DFS)")
print("Graph (adjacency list):")
for k, v in graph.items():
    print(f"  {k} -> {v}")
print("\nStart node: A\n")
order = dfs(graph, "A", [])
print("\nDFS traversal order:", " -> ".join(order))
```

**Output:**

Output - p8_dfs.py

```
DEPTH FIRST SEARCH (DFS)
Graph (adjacency list):
  A -> ['B', 'C']
  B -> ['A', 'D', 'E']
  C -> ['A', 'F']
  D -> ['B']
  E -> ['B', 'F']
  F -> ['C', 'E']

Start node: A

Visiting: A
  Visiting: B
    Visiting: D
    Visiting: E
      Visiting: F
        Visiting: C

DFS traversal order: A -> B -> D -> E -> F -> C
```

**Result:** The Python program for Depth First Search (DFS) was executed successfully and the output was verified.

# Program 9: Travelling Salesman Problem

**Course Outcome:** CO4

**Aim:** To write a Python program to find the shortest route that visits every city exactly once and returns to the starting city.

**Program:**

```
from itertools import permutations

cities = ["A", "B", "C", "D"]
dist = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0],
]


def tsp(start=0):
    n = len(cities)
    others = [i for i in range(n) if i != start]
    best_cost, best_route = float("inf"), None
    for perm in permutations(others):
        route = [start] + list(perm) + [start]
        cost = sum(dist[route[i]][route[i + 1]] for i in range(n))
        print("Route:", " -> ".join(cities[i] for i in route), "| Cost =", cost)
        if cost < best_cost:
            best_cost, best_route = cost, route
    return best_cost, best_route


print("TRAVELLING SALESMAN PROBLEM (Brute Force)")
print("Distance matrix:")
print("    " + "  ".join(f"{c:>2}" for c in cities))
for i, row in enumerate(dist):
    print(f"{cities[i]:>2}  " + "  ".join(f"{d:>2}" for d in row))
print("\nAll possible tours starting from A:\n")
cost, route = tsp()
print("\nShortest route:", " -> ".join(cities[i] for i in route))
print("Minimum cost  :", cost)
```

**Output:**

Output - p9_tsp.py

```
TRAVELLING SALESMAN PROBLEM (Brute Force)
Distance matrix:
     A   B   C   D
 A   0  10  15  20
 B  10   0  35  25
 C  15  35   0  30
 D  20  25  30   0

All possible tours starting from A:

Route: A -> B -> C -> D -> A | Cost = 95
Route: A -> B -> D -> C -> A | Cost = 80
Route: A -> C -> B -> D -> A | Cost = 95
Route: A -> C -> D -> B -> A | Cost = 80
Route: A -> D -> B -> C -> A | Cost = 95
Route: A -> D -> C -> B -> A | Cost = 95

Shortest route: A -> B -> D -> C -> A
Minimum cost  : 80
```

**Result:** The Python program for Travelling Salesman Problem was executed successfully and the output was verified.

# Program 10: A* Algorithm

**Course Outcome:** CO4

**Aim:** To write a Python program to find the least-cost path from a start node to a goal node using the A* algorithm.

**Program:**

```
import heapq

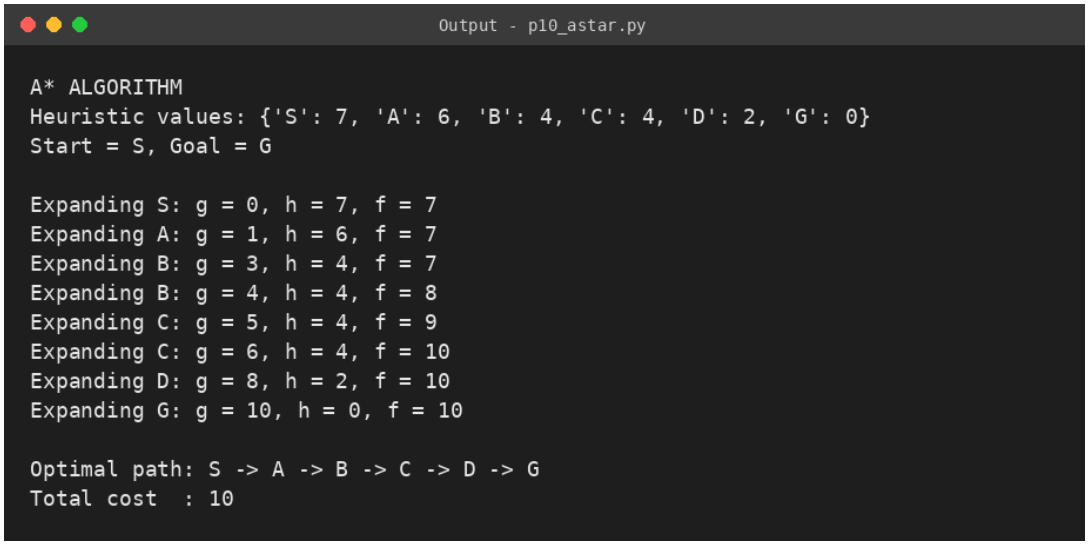
graph = {
    "S": {"A": 1, "B": 4},
    "A": {"B": 2, "C": 5, "D": 12},
    "B": {"C": 2},
    "C": {"D": 3, "G": 7},
    "D": {"G": 2},
    "G": {},
}

# Heuristic: estimated cost from each node to goal G (admissible)
h = {"S": 7, "A": 6, "B": 4, "C": 4, "D": 2, "G": 0}


def a_star(start, goal):
    open_list = [(h[start], 0, start, [start])]
    closed = set()
    while open_list:
        f, g, node, path = heapq.heappop(open_list)
        print(f"Expanding {node}: g = {g}, h = {h[node]}, f = {f}")
        if node == goal:
            return path, g
        if node in closed:
            continue
        closed.add(node)
        for nb, cost in graph[node].items():
            if nb not in closed:
                ng = g + cost
                heapq.heappush(open_list, (ng + h[nb], ng, nb, path + [nb]))
    return None, float("inf")


print("A* ALGORITHM")
print("Heuristic values:", h)
print("Start = S, Goal = G\n")
path, cost = a_star("S", "G")
print("\nOptimal path:", " -> ".join(path))
print("Total cost  :", cost)
```

**Output:**




**Result:** The Python program for A* Algorithm was executed successfully and the output was verified.